ISAILO MAPS

# guía completa del proyecto

Como se organiza, como viajan los datos y como explicar el código más complejo

guía de lectura técnica y funcional
Instituto San Ignacio de Loyola
Octubre de 2026



Explicacion basada en los archivos y comportamientos que existen actualmente en el repositorio.

# Como usar esta guía

Este documento explica el comportamiento que se encuentra en el proyecto, no una arquitectura idealizada. Cuando un comentario no coincide con lo que ejecutan las rutas, se señala para evitar confusiones al presentar el sistema.

| Marca | Significado |
|---|---|
| En palabras sencillas | Idea que sirve para explicar sin leer todo el código. |
| Recorrido | Archivos que participan para completar una accion. |
| Fragmento | código corto con una explicacion de su función. |
| Ojo | Detalle del código actual que conviene tener presente. |

## Contenido


- 1. Que hace la aplicación y como se conectan sus piezas
- 2. Mapa de archivos y rutas
- 3. Flask, inicio de sesión y permisos
- 4. Base de datos y relaciones
- 5. Eventos y calendario público
- 6. Solicitudes y ocupacion de espacios
- 7. Panel.js: estado y estructura
- 8. Asincronia, navegacion y delegacion de eventos
- 9. Graficos SVG, plantillas y animaciones
- 10. Calendario público, pestañas y espacios
- 11. Arranque, problemas frecuentes y guía de cambios
- 12. Glosario y explicacion para una exposicion


IMPORTANTE: Las plantillas deben abrirse desde Flask. Un archivo file:// no procesa Jinja, no tiene las rutas /api y no se conecta con MySQL.

# 1. Que hace ISAILO MAPS

ISAILO MAPS es una aplicación web escolar para organizar espacios, solicitudes, calendario y usuarios. Flask sirve las páginas y MySQL conserva los datos.

Cuando una persona usa un boton, el navegador ejecuta JavaScript. El JavaScript envia una petición a una URL. Flask encuentra la ruta, comprueba permisos, llama al controlador, que aplica reglas de negocio, y este utiliza un modelo para consultar o modificar la base de datos. Flask devuelve una respuesta, normalmente JSON, y el navegador actualiza la interfaz.

| Capa | Responsabilidad | Ejemplo |
|---|---|---|
| Interfaz | Pinta páginas, formularios y botones. | templates/*.html, static/*.css, static/*.js |
| Ruta/API | Recibe URL y método; aplica proteccion. | routes/evento_routes.py |
| Controlador | válida reglas de negocio y decide resultado. | EventoController en controllers/controller.py |
| Modelo | Ejecuta SQL y convierte filas. | models/evento_model.py |
| Base de datos | Guarda información persistente. | MySQL |

## Camino de una accion

| Paso | Que sucede |
|---|---|
| 1. Interfaz | La persona pulsa un boton o envia un formulario. |
| 2. JavaScript | Lee los campos y prepara fetch. |
| 3. Ruta | Flask comprueba sesión y rol; recibe JSON. |
| 4. Controlador | válida campos y reglas del negocio. |
| 5. Modelo | Ejecuta SQL parametrizado. |
| 6. Respuesta | Flask devuelve JSON y la interfaz se repinta. |

Navegador -> fetch(/api/...) -> Ruta -> Controlador -> Modelo -> MySQL
Navegador <- JSON <- Flask <- resultado validado <- filas SQL

IDEA CLAVE: La ruta no deberia contener todo el negocio. El modelo tampoco deberia decidir que rol tiene permiso. Separar responsabilidades facilita revisar y explicar.

# 2. Mapa de archivos y rutas

| Archivo o carpeta | función en el proyecto |
|---|---|
| app.py | Crea Flask, carga Config, ejecuta init_db() y registra blueprints. |
| config.py | Lee SECRET_KEY y variables DB_HOST, DB_PORT, DB_USER, DB_PASSWORD y DB_NAME. |
| routes/ | URLs de páginas y API para clientes, espacios, eventos, panel y solicitudes. |
| controllers/controller.py | Contiene clases centrales usadas por las rutas actuales. |
| controllers/*_controller.py | Hay modulos especializados en paralelo; confirmar import real antes de editar. |
| models/ | Consultas SQL de usuarios, espacios, eventos y solicitudes. |
| database/conexión.py | Conecta MySQL y crea o migra tablas al iniciar. |
| utils/auth.py | Decoradores de login y rol. |
| templates/ | páginas Jinja. |
| templates/partials/ | Piezas reutilizables, como cabecera o barra lateral. |
| static/panel.js | interacción del panel: vistas, cargas, acciones, tablas y graficos. |
| static/js/motion.js | Interfaz compartida de animacion y desplazamiento. |
| static/js/calendario-público.js | Pestanas y calendario cultural de la portada. |

## URLs importantes

| URL | Acceso | Resultado |
|---|---|---|
| / | público | Portada. |
| /login | público | Ingreso de estudiantes y docentes. |
| /admin/login | público | Ingreso administrativo. |
| /admin/panel | Administrador | Panel completo. |
| /calidad/panel | Gestion de Calidad | Panel con opciones reducidas. |
| /maestro/panel | Docente | Panel docente. |
| /estudiante/panel | Estudiante | Panel estudiantil. |

OJO: Hay controladores especializados y también clases dentro de controllers/controller.py. Las rutas actuales importan de este último. Busca el import desde routes/ antes de cambiar reglas, para no editar código que esa ruta no ejecuta.

# 3. Flask, inicio de sesión y permisos

Flask resuelve cada petición por URL. Los Blueprints agrupan rutas relacionadas. Iniciar sesión crea un estado recordado por el servidor; cada ruta protegida lo revisa nuevamente.

## Arranque

```
app = Flask(__name__)
app.config.from_object(Config)
init_db()

app.register_blueprint(clientes_bp)
app.register_blueprint(espacios_bp)
app.register_blueprint(eventos_bp)
app.register_blueprint(solicitudes_bp)
app.register_blueprint(panel_bp)
```

init_db() se ejecuta durante el arranque. Las migraciones simples de database/conexión.py se aplican al iniciar. Un Blueprint registra un grupo de rutas en la aplicación Flask.

## Recorrido del login

| Archivo | función |
|---|---|
| templates/login.html | Dibuja el formulario. |
| static/js/login.js | Envia los datos a /api/clientes/login. |
| routes/clientes_routes.py | Recibe la petición y llama al controlador. |
| controllers/controller.py | Busca cuenta, verifica clave y crea la sesión. |
| models/cliente_model.py | Consulta el usuario y comprueba el hash. |

```
@role_required('Administrador', 'Gestion de Calidad', api=True)
def crear_evento():
    datos = request.get_json(silent=True) or {}
    return jsonify(EventoController.crear(datos, session.get('id_usuario')))
```

El decorador corre antes de la función: exige id_usuario en la sesión y comprueba el nombre del rol. Para API devuelve 401 si falta login y 403 si el usuario esta conectado pero su rol no tiene permiso.

## autenticación y autorización

- autenticación: quien esta conectado; se identifica con session.id_usuario.
- autorización: que puede hacer; se determina con session.nombre_rol y reglas del servidor.
- Ocultar un boton solo mejora la interfaz: la seguridad real esta en las rutas y controladores.

| Rol | Alcance que se ve en plantillas y rutas |
|---|---|
| Administrador | Panel completo, gestion de roles y acciones administrativas. |
| Gestion de Calidad | Panel reutilizado con secciones y operaciones restringidas. |
| Docente | Espacios asignados, solicitudes y herramientas docentes. |
| Estudiante | Panel propio, eventos y solicitudes según API. |

# 4. Base de datos y relaciones

| Tabla | Datos | relación principal |
|---|---|---|
| usuarios | Nombre, correo, hash, estado y rol. | Pertenece a roles; puede estar a cargo de espacios. |
| roles | Nombre y descripción del rol. | usuarios.id_rol -> roles.id_rol. |
| espacios | Nombre, tipo, capacidad, estado y encargado. | Se relaciona con solicitudes y eventos. |
| solicitudes | Solicitante, espacio, fecha, horas, actividad y estado. | FK a usuarios y espacios. |
| eventos_institucionales | Nombre, descripción, fechas, color y tipo_evento. | Tiene una o varias sedes. |
| evento_espacios | Pares id_evento/id_espacio. | Tabla puente muchos-a-muchos. |

## Por que evento_espacios

Un festival puede usar auditorio y patio; cada espacio puede alojar diferentes eventos. Una sola columna de espacio dentro del evento no representa esa relación. La tabla puente guarda una fila por cada pareja.

```
eventos_institucionales 1 --- n evento_espacios n --- 1 espacios

Ejemplo: (evento 12, espacio 3) y (evento 12, espacio 7)
```

## migración e inicialización

database/conexión.py crea las tablas y comprueba columnas antes de agregarlas. El cambio del calendario público agrego tipo_evento y evento_espacios. tipo_evento tiene valor predeterminado Institucional para registros anteriores.

```
SHOW COLUMNS FROM eventos_institucionales LIKE 'tipo_evento'
if cursor.fetchone() is None:
    cursor.execute(
        "ALTER TABLE eventos_institucionales ADD COLUMN tipo_evento VARCHAR(30) NOT NULL
DEFAULT 'Institucional'"
    )
```

> OJO: Los eventos antiguos no aparecen por defecto en el calendario cultural. Hay que marcarlos como Cultural y asignarles al menos un espacio desde el panel.

La creacion/actualizacion del evento y sus espacios se confirma en una transacción. conn.commit() hace permanentes los cambios; conn.rollback() revierte todos si algo falla.

# 5. Eventos: formulario, validación y calendario público

| Etapa | Que hace |
|---|---|
| Formulario | admin_panel.html permite definir tipo, nombre, fechas, color, descripción y varios espacios. |
| panel.js | Lee campos y crea arreglo id_espacios; envia POST o PUT. |
| Ruta | evento_routes.py limita escritura a Administrador y Gestion de Calidad. |
| Controlador | válida campos, categoría, IDs existentes y sede obligatoria para eventos culturales. |
| Modelo | Guarda fila principal y pares evento/espacio en MySQL. |

```
cuerpo = {
    'nombre': $('#evNombre').value.trim(),
    'fecha_inicio': $('#evInicio').value,
    'tipo_evento': $('#evTipo').value,
    'id_espacios': Array.from(
       $('#evEspacios').selectedOptions
    ).map(function (opcion) { return Number(opcion.value); })
}
```

El navegador entrega valores de select como texto; Number los convierte en IDs numericos. El servidor repite la validación porque cualquier persona puede alterar una petición desde el navegador.

```
@eventos_bp.route('/api/publico/eventos-culturales', methods=['GET'])
def listar_eventos_culturales_publicos():
    eventos = EventoModel.listar(tipo_evento='Cultural')
    return jsonify({'ok': True, 'eventos': eventos})
```

La consulta pública no usa login requerido. Solo lee eventos clasificados Cultural. Las rutas POST/PUT/DELETE siguen protegidas, así que visitante puede leer pero no gestionar eventos.

## JOIN y GROUP_CONCAT

```
SELECT ev.*,
       GROUP_CONCAT(DISTINCT es.nombre SEPARATOR ', ') AS espacios_nombres
FROM eventos_institucionales ev
LEFT JOIN evento_espacios ee ON ee.id_evento = ev.id_evento
LEFT JOIN espacios es ON es.id_espacio = ee.id_espacio
WHERE ev.tipo_evento = %s
GROUP BY ev.id_evento
```

LEFT JOIN conserva el evento aunque falte relación. GROUP_CONCAT junta varias sedes en un texto. DISTINCT evita repetir nombres. El filtro tipo_evento esta parametrizado en la consulta, no concatenado como dato.

# 6. Solicitudes y ocupacion de espacios

Una solicitud es una petición de uso, no una reserva confirmada. Solo una solicitud aprobada cuenta como ocupacion.

| Estado | Significado |
|---|---|
| pendiente | Espera revisión. |
| aprobada | Ocupa el espacio durante el horario aprobado. |
| rechazada | No ocupa el espacio. |

## Recorrido tipico

```
POST /api/solicitudes
  -&gt; role_required(Docente)
  -&gt; SolicitudController.crear(...)
  -&gt; validación de fecha, espacio y cruces
  -&gt; SolicitudModel.crear(...)
  -&gt; INSERT en MySQL

Revision: PATCH /api/solicitudes/&lt;id&gt;/estado
```

SolicitudModel.ocupacion() construye filtros por rango, espacio y tipo. El texto SQL fijo se compone en código, mientras los valores se envian como parametros para que la base los trate como datos.

```
condiciones = ["s.estado = 'aprobada'", "s.fecha_uso BETWEEN %s AND %s"]
parametros = [desde, hasta]
if id_espacio:
    condiciones.append('s.id_espacio = %s')
    parametros.append(id_espacio)
```

## Como explicar el cruce de horarios

Dos intervalos se cruzan cuando inicioA < finB y finA > inicioB. La comparacion detecta intersección sin recorrer cada minuto. El controlador decide si se puede aprobar; el modelo consulta registros aprobados.

> IDEA CLAVE: El modelo responde que filas cumplen una consulta. El controlador decide si una accion y sus datos respetan las reglas del sistema.

Las rutas también filtran lo que puede ver cada rol. Por ejemplo, los docentes se limitan a espacios a su cargo; el estudiante no obtiene la lista general de solicitudes.

# 7. Panel.js: estructura y memoria compartida

static/panel.js coordina varias funciones de panel y tiene 2.384 líneas en la versión revisada. Esta dentro de una IIFE para mantener la mayoria de variables privadas y evitar contaminar window.

```
(function () {
    // constantes, estado y funciones del panel
})();
```

IIFE quiere decir que la función se invoca inmediatamente. La navegacion se expone intencionalmente con window.showSection; otros nombres quedan encerrados en el archivo.

## El objeto estado

estado es la memoria temporal de la interfaz. Conserva listas cargadas, filtros, IDs que se editan, fechas y vista seleccionada. cargar... suele pedir datos y actualizar estado; pintar... suele dibujar lo que ya esta en estado.

```
estado = {
    espacios: [], eventos: [], usuarios: [], solicitudes: [],
    eventoEditandoId: null, espacioEditandoId: null,
    // filtros, fechas y controles actuales
}
```

| función | Responsabilidad |
|---|---|
| cargarEspacios() | Pide espacios a la API y guarda en estado. |
| pintarEspacios() | Construye la lista o tabla visible. |
| cargarEventos() | Carga eventos y actualiza calendario/tabla. |
| pintarCalPared() | Dibuja calendario anual administrativo. |
| guardarEvento() | Lee formulario y envia POST o PUT. |

## Atajos del DOM

```
function $(sel, ctx) { return (ctx || document).querySelector(sel); }
function $$(sel, ctx) {
    return Array.prototype.slice.call((ctx || document).querySelectorAll(sel));
}
```

$ obtiene el primer elemento que coincide con selector; $$ devuelve una lista de elementos. ctx permite limitar la busqueda a una parte del documento.

## Escape de texto con esc()

```
function esc(texto) {
    var div = document.createElement('div');
    div.textContent = texto == null ? '' : String(texto);
    return div.innerHTML;
}
```

Convierte texto a contenido HTML seguro: por ejemplo, < se transforma en <. Se usa al construir tablas y tarjetas con datos del servidor. Evita interpretar un nombre como si fuera código HTML.

> IDEA CLAVE: La parte difícil no es una función aislada: varios modulos leen y modifican el mismo estado. Para entender uno, sigue la carga, la actualizacion de estado, el dibujo y la accion del usuario.

# 8. Asincronia y arranque del panel

fetch devuelve una promesa porque el servidor responde más tarde. Mientras espera, el navegador puede seguir atendiendo la página. El resultado se procesa con then y los errores con catch.

```
api('/api/eventos').then(function (res) {
    estado.eventos = res.ok ? (res.eventos || []) : [];
    pintarCalPared();
});
```

## Arranque protegido

Al ocurrir DOMContentLoaded, panel.js prepara menu, pestañas, modales, botones y scroll. Luego lanza cargas iniciales y agrega otras dependiendo del rol. seguro(fn, etiqueta) protege cada inicializador para que un fallo no impida ejecutar los demas.

```
var cargas = [cargarEspacios(), cargarPanel()];
// Se agregan cargas según rol
Promise.all(cargas)
  .catch(function (error) { mostrarErrorCarga(); })
  .then(function () { entradaInicial(); });
```

- Promise.all espera a que todas las cargas terminen.
- catch muestra un aviso/reintento si falla la carga.
- Una respuesta HTTP y una promesa son distintas: un 401 puede ser una respuesta recibida, no necesariamente un fallo de red.
- El orden se controla con then/catch; no solo por el orden visual del archivo.

## Navegacion showSection(id)

showSection encuentra la vista nueva, cancela una animacion anterior, limpia estilos residuales, actualiza menu y activa la vista. La ultima petición gana: dos clics rapidos no deben dejar dos vistas visibles.

```
function showSection(id) {
    var nueva = document.getElementById(id);
    if (!nueva) return null;
    if (MOTION &amp;&amp; MOTION.cancelarTransicion) MOTION.cancelarTransicion();
    actualizarMenu(id);
    // Cambiar a la vista y revelar su contenido
}
```

IDEA CLAVE: Las animaciones mejoran la presentacion, pero no son la fuente de verdad. Si Motion no existe o se reduce movimiento, el cambio de vista debe seguir funcionando.

# 9. Eventos dinamicos, graficos y plantillas

## Delegacion de eventos

Las listas se reconstruyen con innerHTML. Enlazar un listener a cada boton antes de volver a dibujar seria fragil: al reconstruir, los botones son elementos nuevos. La delegacion coloca un listener en un padre estable y averigua que boton hijo se pulso.

```
document.addEventListener('click', function (ev) {
    var boton = ev.target.closest('[data-accion]');
    if (!boton) return;
    var accion = boton.getAttribute('data-accion');
    var id = Number(boton.getAttribute('data-id'));
    // ejecutar la accion adecuada
});
```

data-accion identifica la operación; data-id identifica el registro. closest() también encuentra el boton si se pulsa el icono que esta adentro.

## Grafico de líneas SVG

pintarSerie calcula escalas horizontales y verticales a partir de los datos. después crea líneas de cuadricula, rutas path y circulos. En SVG, el atributo d de path es una receta geométrica para dibujar un trazado.

```
Cantidad del día -&gt; posicion X
Valor -&gt; posicion Y
Puntos x/y -&gt; comandos M/L en la ruta SVG
Cada observacion -&gt; circulo SVG
```

## Grafico de dona SVG

pintarDona suma los valores, calcula la proporción de cada categoría y convierte esas fracciones en segmentos de círculo. stroke-dasharray y stroke-dashoffset controlan la parte coloreada. La leyenda y el dibujo deben partir de los mismos datos.

## Cambios que pueden romper esta zona

- Proteger denominadores y valores maximos para que una serie vacía no divida por cero.
- Una accion data-accion nueva necesita un caso en el listener delegado.
- Escapar todo dato que se inserta como texto HTML.
- Cambiar un ID requiere buscar referencias en plantilla, CSS y JavaScript.

## Jinja y HTML

```
{% if es_admin %}
  ...contenido exclusivo...
{% endif %}
{{ session['nombre'] }}
```

{% ... %} ejecuta lógica de plantilla; {{ ... }} imprime valores. Flask procesa Jinja antes de enviar el HTML. Ocultar una seccion no sustituye la validación de acceso en la ruta.

| Hoja de estilos | Uso |
|---|---|
| static/css.css | Portada pública. |
| static/css/tokens.css | Variables de colores y diseno. |
| static/admin.css | Estructura del panel administrativo. |
| static/css/calidad.css | Componentes de panel compartidos. |
| static/css/login.css | Pantallas de acceso. |

# 10. Motion y accesibilidad

static/js/motion.js concentra animaciones y scroll. Expone window.Motion para que otros archivos usen una interfaz común sin saber los detalles de la libreria de animacion.

```
var MOTION = (window.Motion &amp;amp;&amp;amp; window.Motion.animado)
    ? window.Motion
    : null;

if (MOTION) MOTION.scrollTop(true);
else window.scrollTo({ top: 0 });
```

panel.js pregunta si Motion existe. Si no esta disponible, utiliza un cambio directo o una alternativa GSAP. motion.js también considera la preferencia de reducir movimiento del sistema operativo.

## Accesibilidad

- prefers-reduced-motion permite omitir animaciones no esenciales.
- aria-label nombra controles; aria-live anuncia cambios; aria-selected refleja pestañas activas.
- type="button" evita enviar formularios por accidente.
- El foco visible y soporte de teclado evitan depender solo del mouse.

# 11. Calendario y espacios públicos

La portada tiene dos pestañas públicas: Calendario y Espacios. La primera permite consultar eventos culturales sin iniciar sesión; la segunda permanece vacía para colocar otro mapa después.

| Archivo | Responsabilidad |
|---|---|
| templates/index.html | Estructura de pestañas, controles, calendario mensual y zona vacía. |
| static/js/calendario-público.js | Gestiona pestañas, mes seleccionado, petición pública y render. |
| static/css.css | Estilos adaptables de calendario, tarjetas y pestañas. |
| routes/evento_routes.py | GET /api/publico/eventos-culturales; solo lectura y sin login. |
| models/evento_model.py | Filtra Cultural y adjunta nombres de espacios. |

## Calendario de un mes

El JavaScript toma el mes actual, calcula el día de inicio y la cantidad de días del mes. Rellena las celdas de lunes a domingo. Un punto dorado indica que hay evento. La agenda inferior muestra fechas, descripción y espacios para leer la ubicacion completa.

```
return inicio &amp;amp;&amp;amp; inicio &lt;= finMes &amp;amp;&amp;amp; fin &gt;= inicioMes;
```

Esta condicion detecta si dos rangos se cruzan. Un evento que empieza el mes anterior y termina este mes también aparece. No exige que las dos fechas queden dentro del mes visible.

## Pestanas

Cada boton tiene data-panel-tab y aria-controls. activarPestana() cambia clase visual, aria-selected y hidden. Eso mantiene sincronizados el estado visual, la accesibilidad y el panel mostrado.

> PASO operativo: Para que los eventos aparezcan: inicia Flask, edita un evento en el panel, elige tipo Cultural, asigna uno o más espacios y guarda. La portada consulta esos registros en MySQL.

# 12. Arranque y diagnostico

MySQL debe estar activo para que la aplicación pueda inicializar y consultar sus tablas. Ejecuta Flask y visita la raiz del servidor, no la plantilla abierta como archivo local.

```
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py

Navegador: http://127.0.0.1:5000/
```

| Sintoma | Primer lugar que revisar |
|---|---|
| página no abre | URL, ruta Flask, Blueprint y decorador. |
| 401 | No se reconoce la sesión; revisa login/cookie. |
| 403 | Existe sesión, pero ese rol no tiene permiso. |
| Lista vacía | JSON, filtros, SQL, conexión y registros. |
| Panel sin datos | Consola, Promise.all, API y mensaje de reintento. |
| Calendario vacío | Tipo Cultural, fechas, espacios y endpoint público. |
| Boton no responde | data-accion/data-ir y manejador en panel.js. |
| Vista invisible | motion.js, prefers-reduced-motion y estado final. |

## Observaciones del código actual

- Import faltante: routes/solicitud_routes.py usa EspacioModel para filtrar consultas de docentes, pero el archivo actual no lo importa. Esa ruta puede producir NameError.
- Comentario desactualizado: el TODO de arranque en panel.js afirma que cargarEventos/cargarSemana no se llaman, pero si se agregan para ciertos roles.
- Controladores duplicados: las rutas importan clases desde controllers/controller.py aunque existan controladores separados. Confirma referencia antes de cambiar.
- configuración: config.py contiene valores convenientes para desarrollo. En produccion, definir SECRET_KEY y credenciales seguras mediante entorno.

# 13. Cambios seguros y mantenimiento

| Quiero cambiar... | Revisar también... |
|---|---|
| Texto de pantalla | Plantilla Jinja correspondiente. |
| Accion de boton | data-accion, panel.js, ruta y permisos. |
| Campo nuevo | HTML, JS, validación, modelo y esquema de datos. |
| Permisos | Decorador de ruta y regla del controlador. |
| Fechas | Formato, reglas, consulta SQL y render del calendario. |
| Grafico | Datos, escalas, SVG y leyenda. |
| Animaciones | motion.js, plantilla y preferencia de movimiento. |
| Consulta pública | Campos que se exponen; solo lectura para datos públicos. |

## Lista mental para explicar cualquier función

```
1. Quien inicia la accion? (click, submit, carga)
2. Que datos lee del DOM?
3. Que ruta recibe la petición?
4. Que sesión y rol se requieren?
5. Que válida el controlador?
6. Que tabla consulta el modelo?
7. Que respuesta vuelve?
8. Que función repinta la interfaz?
```

- cargar... normalmente pide o prepara datos.
- pintar... normalmente transforma datos en HTML.
- abrirModal... prepara y muestra un formulario emergente.
- data-id identifica el registro; data-accion indica lo que se quiere hacer.

> IDEA CLAVE: El navegador puede alterar cualquier dato que envia. El servidor debe validar de nuevo incluso cuando el formulario ya tiene required o restricciones HTML.

# 14. Glosario y explicacion para exponer

| Termino | Explicacion sencilla |
|---|---|
| Blueprint | Grupo organizado de rutas Flask. |
| Endpoint | Direccion y método que ejecutan una función. |
| API | Intercambio de datos entre navegador y servidor, normalmente JSON. |
| Controlador | válida petición y reglas del negocio. |
| Modelo | Lee o guarda datos en MySQL. |
| sesión | Identidad que el servidor recuerda entre peticiones. |
| Decorador | función que envuelve otra y revisa el acceso antes de ejecutarla. |
| Promesa | Resultado que termina más tarde, como una petición fetch. |
| Delegacion | Un listener padre detecta acciones de botones dinamicos. |
| SVG | Formato vectorial que el panel usa para dibujar graficos. |
| JOIN | Combina datos de tablas relacionadas. |
| transacción | Grupo de escrituras que se confirma o se revierte completo. |
| migración | Cambio controlado al esquema de una base existente. |
| Escape HTML | Convierte texto para que no se interprete como etiquetas. |

## Explicacion oral corta

RESUMEN PARA EXPOSICION: ISAILO MAPS es una aplicación Flask que organiza espacios, solicitudes y eventos en MySQL. Las plantillas muestran cada panel y el JavaScript controla la interacción. Las rutas protegen acciones según sesión y rol; los controladores validan reglas y los modelos ejecutan SQL. La parte más compleja es panel.js porque coordina datos asincronos, estado compartido, vistas dinámicas y graficos SVG. La portada permite consultar eventos culturales y sus espacios sin abrir la gestion administrativa.

## Archivos de referencia

- app.py - arranque Flask y registro de rutas.
- utils/auth.py - sesión y permisos.
- controllers/controller.py - lógica central usada por las rutas.
- models/ - consultas y escrituras MySQL.
- database/conexión.py - conexión y migraciones.
- static/panel.js - lógica del panel.
- static/js/motion.js - animacion y scroll.
- static/js/calendario-público.js - pestañas y calendario público.
- templates/ - estructura HTML/Jinja.